

Mechanics of Composite Materials

Project 3

Luncheng Zhang

Kiefer Gonsalves

Word Count: 421 words

Introduction:

This study analyzes the structural and weight efficiency of three materials — AS4/3501-6 epoxy, Aramid 49/epoxy, and 2024-T3 aluminum alloy — for use in cylindrical pressure vessels. The objective is to determine the best material based on mechanical stress response and mass per surface area. In order to properly analyze these materials we will first be using OSULaminates in order to find each material's stress and strain throughout the laminate. Then using the Tsai-Wu failure criterion we can use Python to interpret our results to find the wall thickness of the laminate and the mass per unit area of each material. Doing so will allow us to find which material has the best performance in terms of each material's ability to withstand stress.

Methods:

For each material, a [+60/−60]s symmetric laminate configuration was defined. The applied mechanical strains ($\epsilon_1 = 0.472\%$, $\epsilon_2 = 0.91\%$) and thermal strain from $\Delta T = -80\text{ }^{\circ}\text{C}$ were used to calculate ply-level stresses using OSULaminates.

Then, the required wall thickness (h_{req}) was computed by programming Python using the Tsai-Wu failure criterion. The stress values σ_1 and σ_2 for each ply were extracted from OSULaminates when setting up the same type of scenario scene in the example problem 9.36. Using these stress values and the material strength properties from the textbook, we applied the Tsai-Wu equation to solve for the wall thickness at which the failure index equals the safety limit ($S_{all} = 2.0$). After computing the required thickness h for each material, a Python script was used to calculate:

Mass per area= $\rho \times h$.

Where ρ is the density in g/cm^3 and h is in cm. The results were plotted in a bar chart showing the comparative weight per surface area of all materials.

Results:

Part One:

Material: AS4/3501-6 Epoxy

Required wall thickness (with safety factor): 2.713 mm

Mass per unit area: 4.26 kg/m²

Part Two:

Material: Aramid 49 Epoxy

Required wall thickness (with safety factor): 3.783 mm

Mass per unit area: 5.45 kg/m²

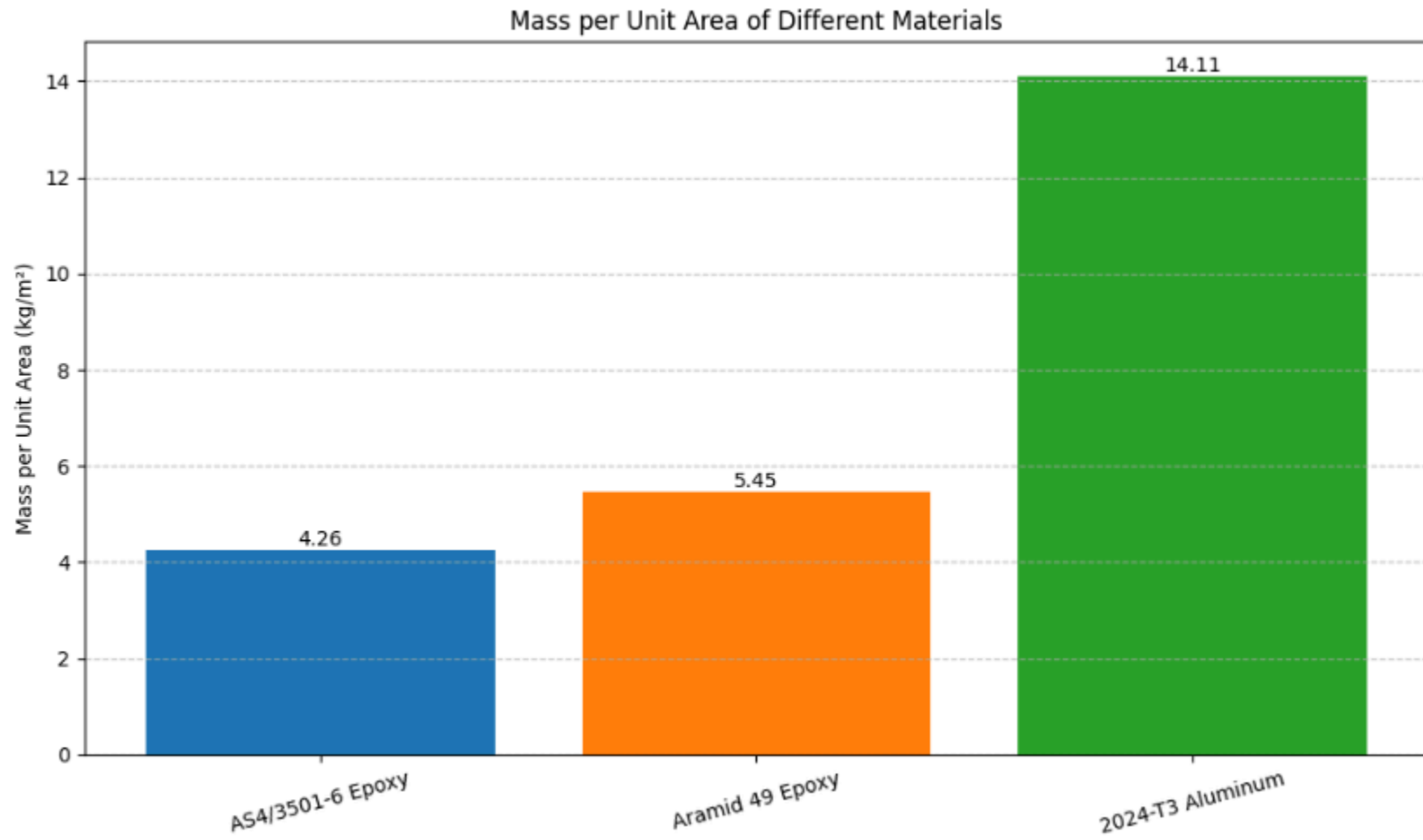
Part Three:

Material: 2024-T3 Aluminum

Required wall thickness (with safety factor): 5.093 mm

Mass per unit area: 14.11 kg/m²

Part Four (Parts 1-3 Graphed):



Conclusion:

Throughout this project we were able to analyze the three different materials in order to determine each material's required wall thickness and mass per square meter. The calculated thickness and OSULaminates stress results showed that AS4/epoxy had the best strength-to-weight performance. Python plots confirmed it required the least material. Compared to aluminum, composite laminates offer significant weight savings, with AS4/epoxy being the most efficient choice. Our second best was the other composite, UD Aramid 49 - epoxy, and the least effective material was the 2024-T3 aluminum. These results indicate that users can save money, time, and material by using composites for their material selection for better performance in scenarios where having a lower mass is critical.

Appendix:

Part 1:

AS4/3501-6 Epoxy

Constants

safety_factor = 2.0

density = 1570 # kg/m³

Material properties

material_name = "AS4/3501-6 Epoxy"

sigma_1_t = 2060 # MPa

sigma_1_c = -1100

sigma_2_t = 78

sigma_2_c = -196

tau_12 = 157

Laminate stresses from OSULaminates (mechanical only)

sigma1 = 143.9227 # MPa

sigma2 = 423.2004 # MPa

tau12 = 0.0 # MPa

Failure index calculation using max stress

def failure_index_max_stress(sx, sy, txy):

 f1 = sx / sigma_1_t if sx >= 0 else sx / sigma_1_c

```
f2 = sy / sigma_2_t if sy >= 0 else sy / sigma_2_c
```

```
f6 = abs(txy) / tau_12
```

```
return max(abs(f1), abs(f2), abs(f6))
```

```
# Calculate failure index at full strain state
```

```
fi = failure_index_max_stress(sigma1, sigma2, tau12)
```

```
# Scale wall thickness so failure index is <= 1/safety factor
```

```
scaling = fi / safety_factor
```

```
h_required_mm = scaling
```

```
mass_per_m2 = h_required_mm / 1000 * density
```

```
# Output results
```

```
print(f"Material: {material_name}")
```

```
print(f"Required wall thickness (with safety factor): {h_required_mm:.3f} mm")
```

```
print(f"Mass per unit area: {mass_per_m2:.2f} kg/m^2")
```

Part 2:

```
# Aramid 49 Epoxy

# Constants

safety_factor = 2.0

density = 1440 # kg/m^3


# Material properties

material_name = "Aramid 49 Epoxy"

sigma_1_t = 1050 # MPa

sigma_1_c = -250

sigma_2_t = 30

sigma_2_c = -75

tau_12 = 60


# Laminate stresses from OSULaminates (mechanical only)

sigma1 = 111.4687 # MPa

sigma2 = 226.9822 # MPa

tau12 = 0.0 # MPa


# Failure index calculation using max stress

def failure_index_max_stress(sx, sy, txy):

    f1 = sx / sigma_1_t if sx >= 0 else sx / sigma_1_c

    f2 = sy / sigma_2_t if sy >= 0 else sy / sigma_2_c
```



```
f6 = abs(txy) / tau_12
```

```
return max(abs(f1), abs(f2), abs(f6))
```

```
# Calculate failure index at full strain state
```

```
fi = failure_index_max_stress(sigma1, sigma2, tau12)
```

```
# Scale wall thickness so failure index is <= 1/safety factor
```

```
scaling = fi / safety_factor
```

```
h_required_mm = scaling
```

```
mass_per_m2 = h_required_mm / 1000 * density
```

```
# Output results
```

```
print(f"Material: {material_name}")
```

```
print(f"Required wall thickness (with safety factor): {h_required_mm:.3f} mm")
```

```
print(f"Mass per unit area: {mass_per_m2:.2f} kg/m^2")
```

Part 3:

2024-T3 Aluminum

Constants

material_name = "2024-T3 Aluminum"

yield_strength = 320 # MPa

density = 2770 # kg/m³

safety_factor = 2.0

Given laminate stresses

sigma1 = 658.7 # MPa

sigma2 = 911.3 # MPa

tau12 = 0.0 # MPa

Von Mises stress calculation

def von_mises(sx, sy, txy):

return ((sx**2 + sy**2 - sx * sy + 3 * txy**2) ** 0.5)

Calculate actual von Mises stress

sigma_vm = von_mises(sigma1, sigma2, tau12)

Scaling required for safety

scaling = sigma_vm / (yield_strength / safety_factor)

h_required_mm = scaling # this acts like a strain-to-thickness analog here

```
mass_per_m2 = h_required_mm / 1000 * density
```

```
# Output
```

```
print(f"Material: {material_name}")
```

```
print(f"Required wall thickness (with safety factor): {h_required_mm:.3f} mm")
```

```
print(f"Mass per unit area: {mass_per_m2:.2f} kg/m^2")
```

Part Four (Parts 1-3 Graphed):

```
import matplotlib.pyplot as plt

# Data for the materials

materials = [

    {"name": "AS4/3501-6 Epoxy", "mass_per_m2": 4.26},

    {"name": "Aramid 49 Epoxy", "mass_per_m2": 5.45},

    {"name": "2024-T3 Aluminum", "mass_per_m2": 14.11}

]


# Extract names and masses

names = [m["name"] for m in materials]

masses = [m["mass_per_m2"] for m in materials]


# Plotting

plt.figure(figsize=(10, 6))

bars = plt.bar(names, masses, color=['#1f77b4', '#ff7f0e', '#2ca02c'])

plt.title("Mass per Unit Area of Different Materials")

plt.ylabel("Mass per Unit Area (kg/m²)")

plt.xticks(rotation=15)

plt.grid(axis='y', linestyle='--', alpha=0.7)


# Add text labels
```

```
for bar in bars:
```

```
    height = bar.get_height()
```

```
    plt.text(bar.get_x() + bar.get_width()/2.0, height, f'{height:.2f}', ha='center', va='bottom')
```

```
plt.tight_layout()
```

```
plt.show()
```